



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Object-Oriented Programming (OOP)

Dawei Chen

Week 4-2, 19 March 2026

Recap – Function

```
def order_summary(price, quantity, *extras, discount_rate=0.1, **info):  
    revenue = price * quantity  
    extra_cost = sum(extras)  
    discount = revenue * discount_rate  
    final_payment = revenue + extra_cost - discount  
    customer_name = info.get("customer", "Unkown")  
    return final_payment, customer_name  
  
result = order_summary(100, 2, 10, 5, discount_rate=0.2, customer="Alice")
```

- (1) What is the data type of `result`? What values does it contain?
- (2) Which arguments are **positional**, **keyword**, `*args`, and `**kwargs`?
- (3) What is the data type of `extras`? What values does it contain?
- (4) What is the data type of `info`? What values does it contain?
- (5) What happens if we call: `order_summary(100, 2)`
- (6) Is the following call valid? Why or why not?

```
order_summary(price=100, quantity=2, 10, 5)
```

Python is an object-oriented language.

In Python, everything is an object,
with its attributes and methods.

Learning Objectives

By the end of today's session, you should be able to:

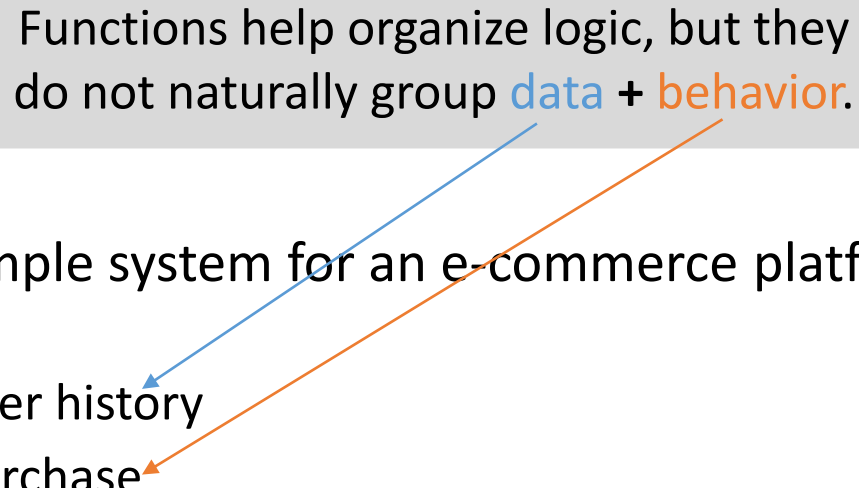
- Understand the concepts of object-oriented programming, class and object
- Define a class and create objects
- Explain and use the `__init__()` method
- Understand the role of the `self` parameter
- Distinguish between object and class attributes
- Access, modify, add, and delete attributes
- Differentiate between object, class, and static methods



Generated by ChatGPT Auto

Warm-up

Functions help organize logic, but they do not naturally group **data** + **behavior**.

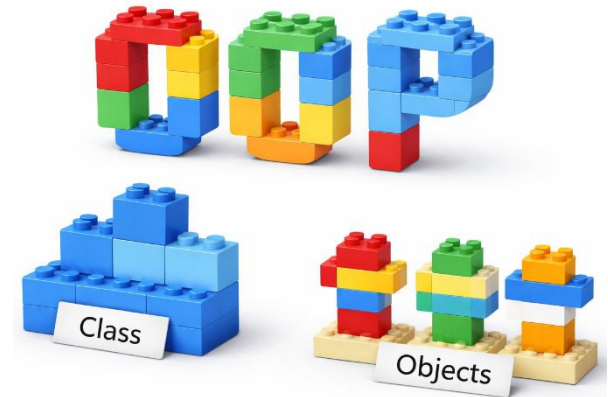


- **Scenario:** You are building a simple system for an e-commerce platform. For each customer, you need to:
 - Store name, balance, and order history
 - Update balance after each purchase
 - Track multiple customers simultaneously
- **Think:** If you use only functions and variables, how would you design this system?
- **Discuss:**
 - Where would you store customer data? (variables? dictionaries?)
 - How do you ensure each customer's data is kept separate?
 - What happens when the system scales to 10,000 customers?
 - Is it easy to maintain and update the code?

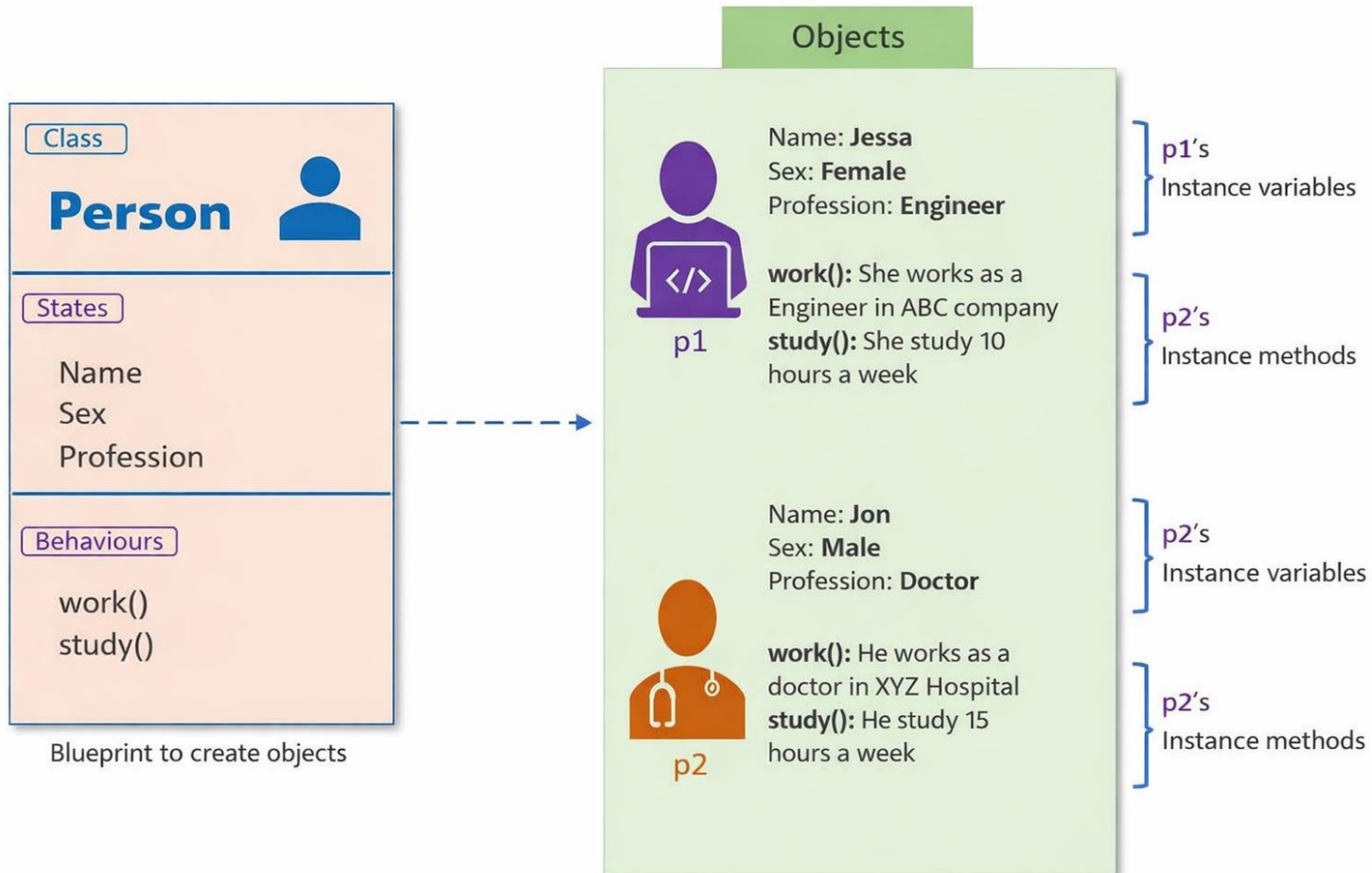
A better way to model real-world entities →
Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) 面向对象编程

- A programming approach that structures code using classes and objects
 - **Class** → blueprint defining data and behavior
 - **Object** → instance created from a class
- Advantages
 - Enables better organization, modularity, and reusability
 - Improves maintainability, and debugging
 - Supports the DRY principle (avoid repeating code)
 - Facilitates building scalable applications



Generated by ChatGPT Auto



Class 类

- A user-defined blueprint or code template for creating objects
- Combines **attributes** (data) and **methods** (functions)
- Used to create multiple objects

Object 对象

- An instance 实例 of a class
- Contains specific **data** (attributes) and can perform **actions** (methods)
- Used to interact with and execute functionality

Source: <https://pynative.com/python-classes-and-objects/>

Create a Class

How is different from defining a function?

```
class Person: # define a class
    species = "Human" # class attributes
    def __init__(self, name1, sex1="Male", profession1):
        # object attributes
        self.name = name1
        self.sex = sex1
        self.profession = profession1

    # object methods
    def show(self):
        print("Name:", self.name, "Sex:", self.sex, "Profession:", self.profession)

    def work(self, company="the company"):
        print(self.name, "is working as a", self.profession, "in", company)

# create an instance of the Person class
Alice = Person("Alice", "Female", "Software Engineer")
# use object methods
Alice.show()
Alice.work("Google")
```


The `__init__` Method

- A **built-in** method that **runs automatically** when a new object is **created**
- Used to **initialize** object **attributes** and perform setup
- Works like a function
 - Accepts positional, keyword, default, `*args`, `**kwargs` arguments
 - First parameter is always `self` (refers to the object)
- Why use it?
 - Ensure consistent object creation
 - Avoid manually assigning attributes after creation

How is `__init__()` different from a general function?

```
class Person:
    def __init__(self, name, sex = "Male", profession):
        # attributes
        self.name = name
        self.sex = sex
        self.profession = profession

Alice = Person("Alice", "Female", "Software Engineer")
```

The `self` Parameter

- Used **within methods defined in a class**, not outside the class, not in standalone functions
- Refers to the current instance (object) of the class
- Enables access to and modify object attributes and methods
- Required as the **first parameter** in instance methods
- **Passed automatically** when a method is called
- A naming **convention** (can be renamed, but should not)

```
def work(self, company="the company"):
    print(self.name, "is working as a", self.profession, "in", company)

Alice = Person("Alice", "Female", "Software Engineer")

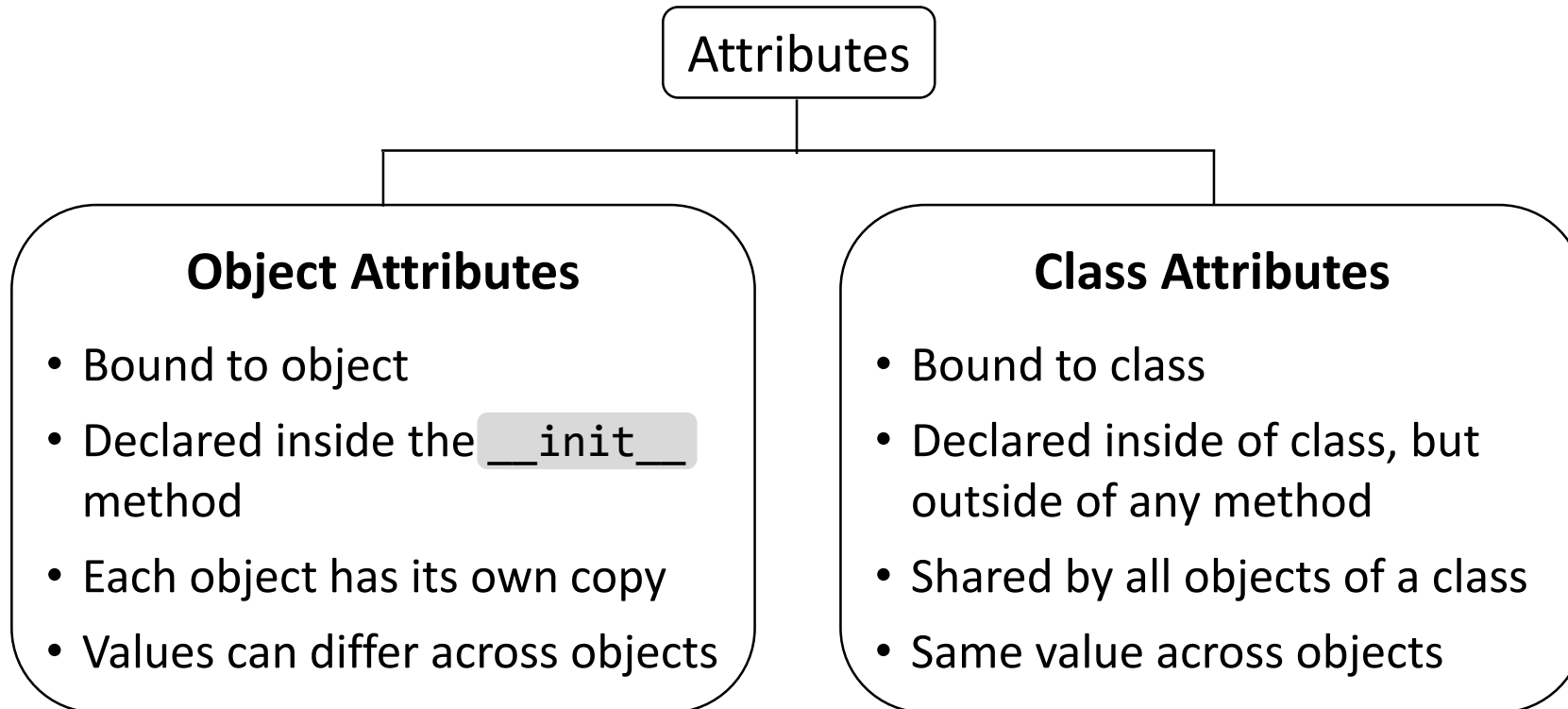
Alice.show()
Alice.work("Google")
```

Attributes

- Attributes are variables that belong to a class.
- They store data for each object created from the class.

```
class Person:
    species = "Human" # class attribute

    def __init__(self, name):
        self.name = name # object attribute
p1 = Person("Alice")
p2 = Person("David")
```



Attributes

Access Attributes

```
Alice.sex  
Person.species
```

Modify Attributes

```
Alice.sex = "Male"  
Person.species = "Animal"
```

Add Attributes

```
Alice.age = 20  
Person.new_attr = "New Value"
```

Delete Attributes

```
del Alice.age  
del Person.new_attr
```

Methods

- Methods are functions defined inside a class
- They define the behavior of objects

Object Methods

- Bound to object
- Access and modify both object and class attributes

Class Methods

- Bound to class
- Can access only class attributes
- Use `@classmethod`

Static Methods

- Independent utility functions
- Do not access object or class attributes
- Use `@staticmethod`

Methods

```
class Person:
    species = "Human"    # class variable
    def __init__(self, name, sex, profession):
        self.name = name
        self.sex = sex
        self.profession = profession

    # object method
    def work(self, company="the company"):
        print(self.name, "is working as a", self.profession, "in", company)

    # class method
    @classmethod
    def show_species(cls):
        print("Species:", cls.species)

    # static method
    @staticmethod
    def is_adult(age):
        return age >= 18
```

Inheritance 继承

- Inheritance allows a class (child) to inherit attributes and methods from another class (parent)
- Promotes code reuse and hierarchical design
- Use `super()` to call parent methods

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person): # inherits from Person
    def __init__(self, name, year):
        super().__init__(name)
        self.year = year
```

Objects and Data Types in Python

- Python supports multiple data types

`1234` (int), `3.14` (float), `"Hello"` (str), `[1, 5, 7]` (list), `{"WKU": "MGS3001"}` (dict)

- Everything is an object, each object has:

- A type $\approx$ class
- An internal representation (data structure / attributes)
- A set of operations (methods / functions)

- Example: `L = [1, 2, 3]`

- **Type:** list (class)
- **Attributes (data):** access, modify, add, delete elements `L[i:j]`
- **Methods (operations):** `L.append()`, `L.remove()`, `L.sort()`

In-Class Exercise

```
class Product:
    category = "General"

    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def display_info(self):
        print(self.name, self.price, self.stock)
```

1) What is the difference between class attribute and object attributes in this code?

2) If we create two objects:

```
p1 = Product("Laptop", 8000, 10)
p2 = Product("Phone", 5000, 20)
```

- Which attributes are shared?
- Which are different? Why?

3) What will happen if we run:

```
p1.category = "Electronics"
print(p1.category, p2.category)
```

Explain the result.

4) What will happen if we run:

```
Product.category = "Electronics"
print(p1.category, p2.category)
```

Explain the difference from Q3.

5) Predict the output

```
p1.display_info()
```

Python is an object-oriented language.

In Python, everything is an object,
with its attributes and methods.

Summary

- **Class** → blueprint that defines attributes (data) and methods (behavior)
- **Object** → instance of a class with its own data
- `__init__()` → initializes object attributes when created
- `self` → refers to the current object
- Attribute
 - Object attributes → unique to each object
 - Class attributes → shared across all objects
- Methods
 - Object methods → operate on object data
 - Class methods → operate on class-level data
 - Static methods → independent utility functions
- Inheritance
 - Enables code reuse and extension
 - Child class inherits from parent class

Q&A

CBPM B214

dchen@kean.edu

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30